Design Specifications

System Architecture & Technical Design

Project: VOO Ward (Kyamatu Ward) Citizen Engagement Platform

Version: 2.0.0

Date: April 2, 2026

Prepared By: VOO Ward Development Team

Design Specifications Document

VOO Ward Citizen Engagement Platform

Field	Detail
Project Name	VOO Ward (Kyamatu Ward) Citizen Engagement Platform
Version	2.0.0
Date	April 2, 2026
Prepared By	VOO Ward Development Team
Document Type	Design Specifications

Table of Contents

1. Introduction

1.1 Purpose

This document provides the detailed design specifications for the VOO Ward Citizen Engagement Platform. It describes the system architecture, module designs, database

schemas, API contracts, security mechanisms, and deployment strategies used to build and maintain the platform.

1.2 Scope

The VOO Ward platform is a multi-channel citizen service system serving the residents of Kyamatu Ward. It enables civic participation through four access channels:

- **USSD** — Accessible to feature phones via short code `*384#`
- **Web Admin Dashboard** — For ward administrators and the Member of County Assembly (MCA)
- **Citizen Portal API** — Self-service REST APIs for citizens
- **WhatsApp Integration** — Conversational access via Twilio webhook
- **Mobile App** — React Native/Expo app for smartphones

1.3 Intended Audience

- Software developers and maintainers
- Academic reviewers and examiners
- Quality assurance engineers
- System administrators and DevOps personnel

1.4 Definitions and Acronyms

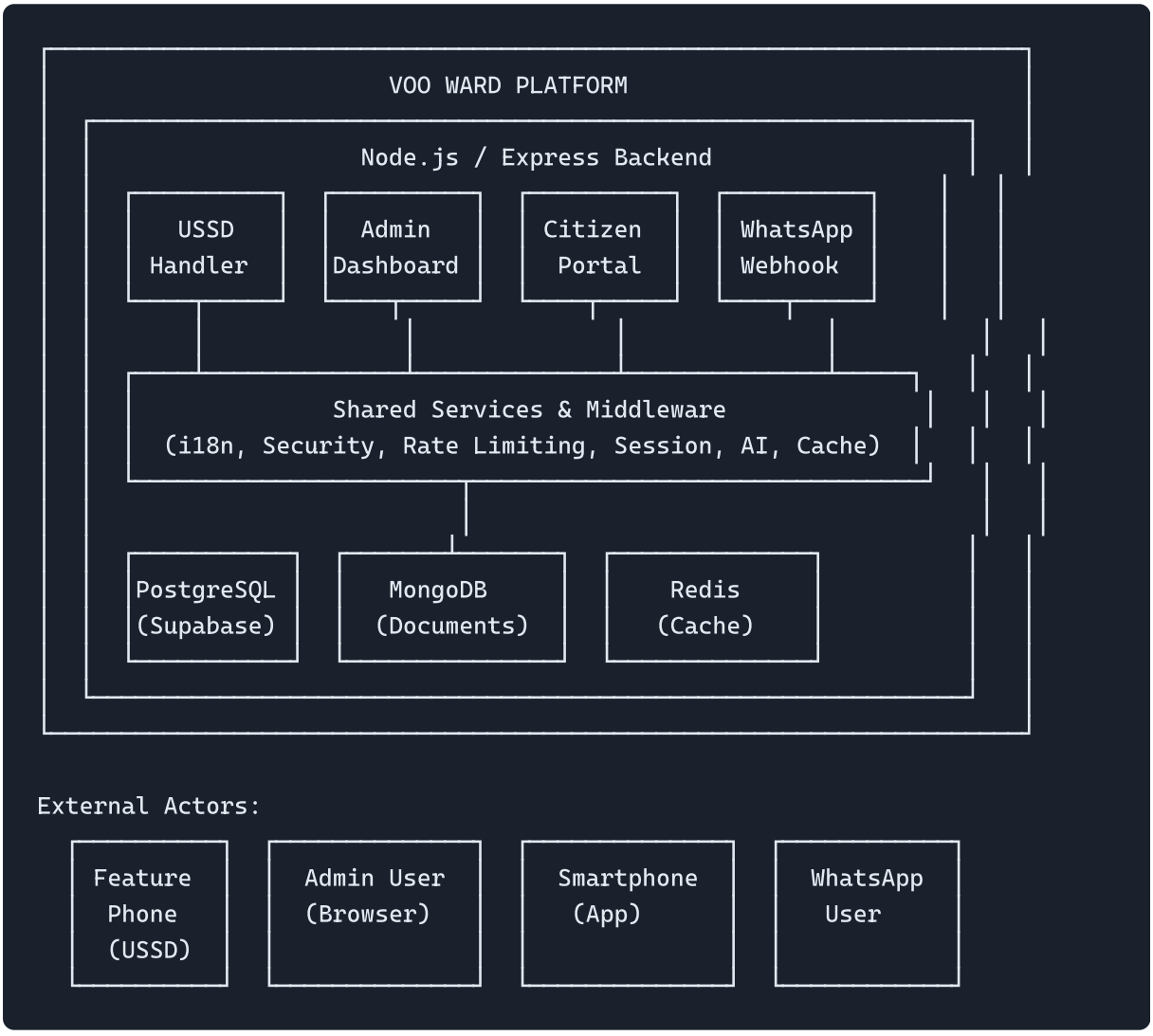
Acronym	Definition
USSD	Unstructured Supplementary Service Data
OTP	One-Time Password
MCA	Member of County Assembly (Super Admin role)
PA	Personal Assistant (Admin role with limited access)
JWT	JSON Web Token
MSISDN	Mobile Station International Subscriber Directory Number
TwiML	Twilio Markup Language
CRUD	Create, Read, Update, Delete
SLA	Service Level Agreement
API	Application Programming Interface

2. System Overview

2.1 Product Perspective

The VOO Ward platform is a backend-centric system built using Node.js and Express.js. It exposes HTTP endpoints consumed by multiple channels and client applications. The system bridges the digital divide by supporting both feature phones (via USSD) and smartphones (via mobile app and WhatsApp).

2.2 System Context Diagram



2.3 Key Design Principles

1. **Multi-Channel Accessibility** — One backend serves USSD, web, mobile, and WhatsApp
2. **Graceful Degradation** — System never crashes; fail-safe messages are returned
3. **Multilingual Support** — English, Swahili, and Kamba language support
4. **Role-Based Access Control** — MCA (super admin) and PA (admin) roles

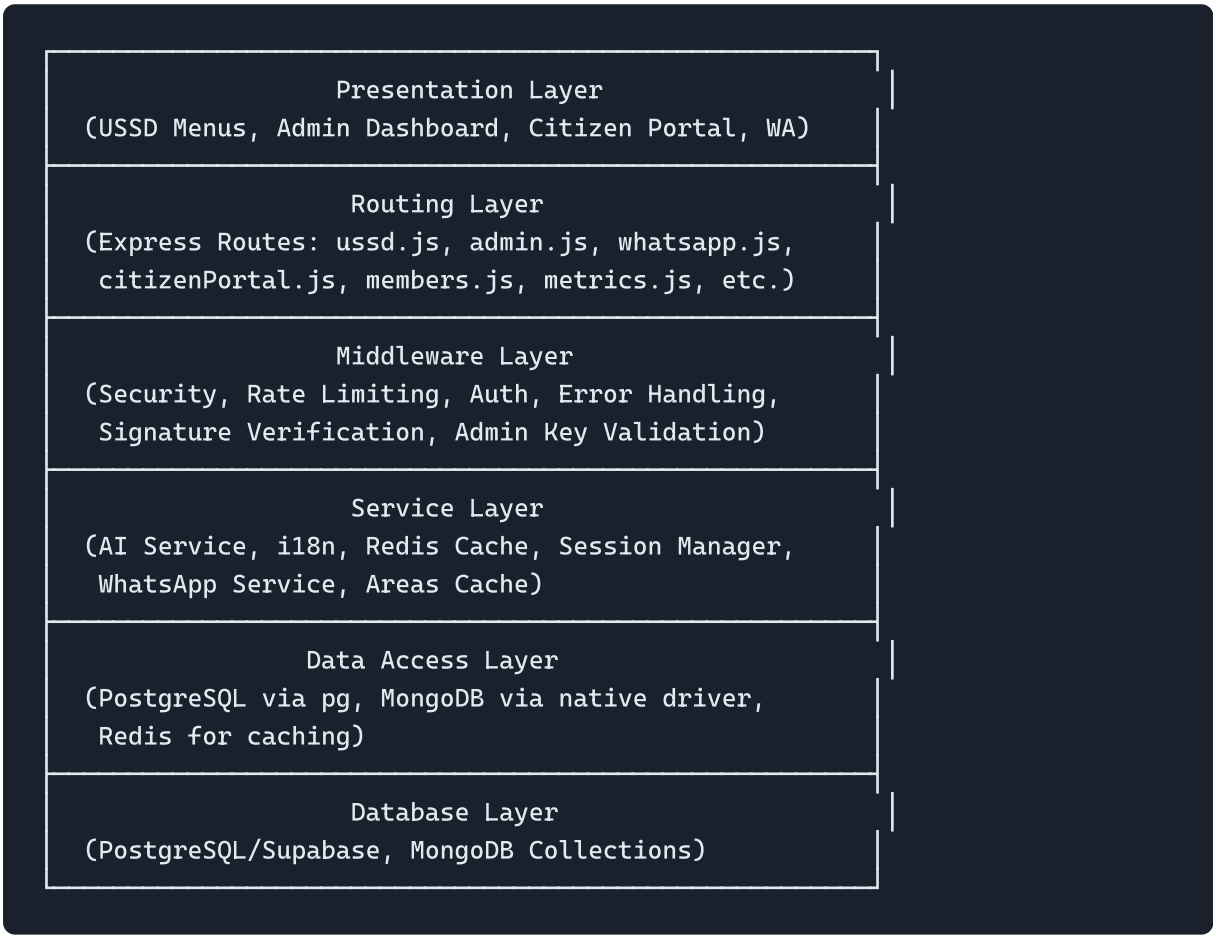
5. **Modular Architecture** — Route, service, middleware, and database layers
6. **Security by Design** — Rate limiting, input sanitization, hashed credentials

---

3. Architecture Design

3.1 Architectural Pattern

The system follows a **layered monolithic architecture** with clear separation of concerns:

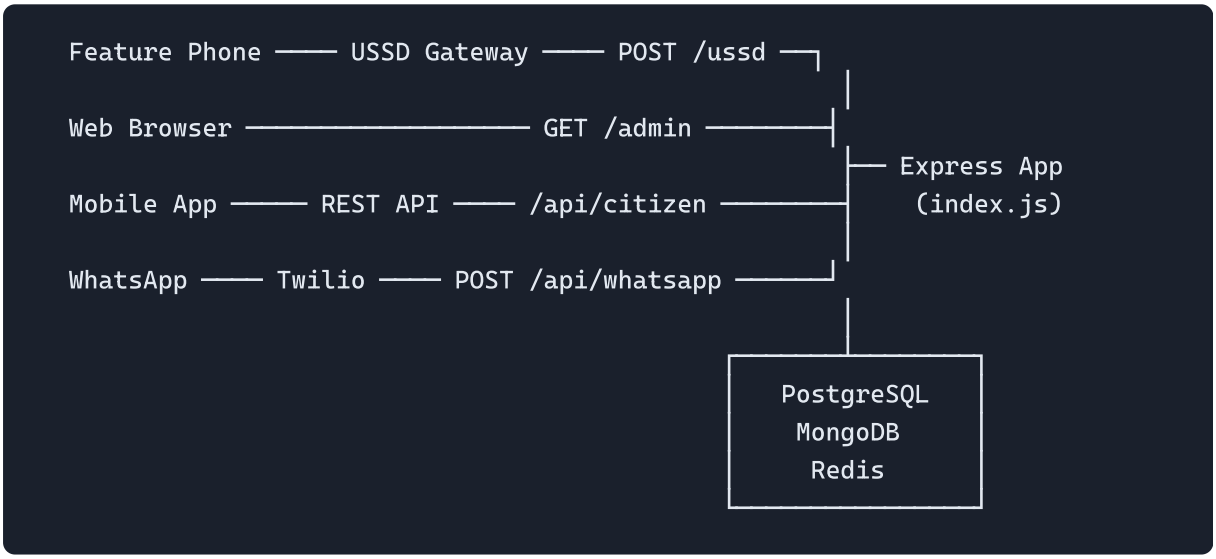


3.2 Directory Structure

```
voo-ward-ussd/
├── src/
│   ├── index.js                # Main Express application entry point
│   ├── admin-dashboard.js      # Admin dashboard (self-contained Express app)
│   ├── ussdCore.js             # Core USSD menu handler
│   ├── ussd-handler.js         # Provider-agnostic USSD router
│   ├── chatbot.js              # AI chatbot with knowledge base
│   ├── ai_assistant.py         # Python-based AI assistant ("Mai")
│   ├── config/                 # Configuration files
│   ├── db/                     # Database helpers and preferences
│   ├── lib/                    # Shared libraries
│   │   ├── cache.js            # Caching utilities
│   │   ├── crypto.js           # Cryptographic utilities
│   │   ├── db.js               # PostgreSQL connection
│   │   ├── mongo.js            # MongoDB connection
│   │   ├── privacy.js          # Data privacy utilities
│   │   ├── rateLimiter.js      # Rate limiting
│   │   ├── utils.js            # General utilities
│   │   └── validators.js       # Input validators
│   ├── middleware/              # Express middleware
│   │   ├── adminKey.js         # Admin API key validation
│   │   ├── errorHandler.js     # Global error handling
│   │   ├── rateLimit.js        # Rate limiting middleware
│   │   ├── security.js         # Security middleware class
│   │   └── signature.js        # Request signature verification
│   ├── routes/                  # API route handlers
│   │   ├── admin.js            # Admin CRUD operations
│   │   ├── adminAreas.js       # Ward area management
│   │   ├── adminExports.js     # CSV data exports
│   │   ├── citizenPortal.js    # Citizen self-service APIs
│   │   ├── members.js          # Member management
│   │   ├── metrics.js          # Analytics and metrics
│   │   ├── mongo-crud.js       # MongoDB CRUD operations
│   │   ├── stats.js            # Statistics endpoints
│   │   ├── textbooks.js        # Textbook management
│   │   ├── ussd.js             # Full USSD route handler (596 lines)
│   │   └── whatsapp.js         # WhatsApp webhook handler
│   ├── services/                # Business logic services
│   │   ├── aiService.js        # OpenAI integration
│   │   ├── areasCache.js       # Ward areas cache
│   │   ├── i18n.js             # Internationalization (EN/SW)
│   │   ├── redisCache.js       # Redis caching service
│   │   ├── sessionManager.js   # Session management
│   │   ├── sessionStore.js     # Session persistence
│   │   └── whatsappService.js  # Twilio WhatsApp service
│   └── utils/                   # Utility functions
├── mobile-app/                  # React Native / Expo mobile application
│   ├── App.js                  # Main app entry
```

```
    └── src/screens/             # Mobile app screens
        ├── LoginScreen.js
        ├── DashboardScreen.js
        ├── ReportIssueScreen.js
        ├── MyIssuesScreen.js
        ├── AnnouncementsScreen.js
        ├── BursaryStatusScreen.js
        ├── VoterRegistrationScreen.js
        └── EnhancedIssueDetailScreen.js
├── migrations/                  # Database migration scripts
├── public/                      # Static web assets
├── scripts/                     # Utility scripts
├── db/                          # Database backup and migration files
└── docs/                        # Project documentation
```

3.3 Component Interaction Diagram



---

4. Module Design

4.1 USSD Module

Purpose: Provides menu-driven access for feature phone users.

Components:

Component	File	Description
Core Handler	`ussdCore.js`	Language selection, main menu, registration, issue reporting
Full Handler	`routes/ussd.js`	Database-integrated USSD flow (596 lines)
Lightweight API	`ussd-handler.js`	Provider-agnostic bursary/news USSD handler

USSD Menu Flow:

```
*384# → Language Selection (English/Swahili/Kamba)
  └── Main Menu
      ├── 1. Register → Full Name → National ID → Confirmation
      ├── 2. Report Issue → Category → Description → Ticket ID
      ├── 3. Announcements → Latest news list
      ├── 4. Projects → Active projects
      ├── 5. Bursary → Apply / Check Status / Requirements
      └── 0. Exit
```

Key Design Decisions:

- Uses `CON` (continue) and `END` (terminate) prefixes for telecom aggregators
- Session data is tracked per `sessionId` provided by the USSD gateway
- Graceful fallback: if DB is unavailable, the user sees a friendly error message
- Three-language support with a localization helper function `L(en, sw, kb)`

4.2 Admin Dashboard Module

Purpose: Full-featured administration panel for MCA and PA users.

File: `admin-dashboard.js` (2,233 lines) — Self-contained Express app mounted at `/admin`

Key Features:

Feature	Description
Authentication	Username + 6-digit PIN with bcrypt hashing
Session Management	Token-based sessions with 30-minute timeout
Announcements CRUD	Create, read, delete ward announcements
Issues Management	View, update status, comment on citizen issues
Constituent Management	View registered citizens, verify identity
Bursary Administration	Review applications, approve/reject, add notes
User Management	MCA can create/delete PA accounts
Data Export	CSV export for constituents, issues, and bursary data
Citizen Messages	View and respond to citizen feedback
Avatar Upload	Profile picture upload with S3 or local storage

Role-Based Access:

Role	Permissions
MCA (Super)	Full access: manage users, verify constituents, all CRUD
PA (Admin)	Limited: manage content, view data, no user management

4.3 Citizen Portal Module

Purpose: Self-service API for authenticated citizens.

File: `routes/citizenPortal.js`

Authentication Flow:

```
1. Citizen requests OTP → POST /api/citizen/request-otp
2. System generates 6-digit OTP (valid 5 minutes)
3. OTP sent via SMS (logged in development)
4. Citizen verifies OTP → POST /api/citizen/verify-otp
5. System returns Bearer token (valid 24 hours)
6. Subsequent requests use Authorization: Bearer <token>
```

Endpoints:

- `POST /api/citizen/request-otp` — Request login OTP
- `POST /api/citizen/verify-otp` — Verify OTP and receive token
- `GET /api/citizen/issues` — View reported issues

- `POST /api/citizen/issues` — Submit new issue
- `GET /api/citizen/bursaries` — View bursary applications

4.4 WhatsApp Integration Module

Purpose: Conversational access to ward services via WhatsApp.

File: `routes/whatsapp.js`

Design:

- Receives Twilio webhook payloads at `POST /api/whatsapp/webhook`
- Maintains in-memory session state per phone number
- Responds with TwiML XML format
- Supports three languages (English, Swahili, Kamba)
- Handles media attachments (photos) for issue reports

4.5 AI Chatbot Module

Purpose: Intelligent assistant ("Mai") for citizen queries.

Files: `chatbot.js`, `chatbot_kb.json`, `services/aiService.js`

Design:

- Knowledge base stored in JSON with Q&A pairs
- OpenAI GPT integration for advanced queries
- Contextual responses based on ward-specific information

4.6 Mobile Application Module

Purpose: Native smartphone app for citizen engagement.

Technology: React Native with Expo framework

Screens:

Screen	Purpose
LoginScreen	OTP-based authentication
DashboardScreen	Feature cards overview (5 features)
ReportIssueScreen	Camera, location, category-based reporting
MyIssuesScreen	View and track reported issues
EnhancedIssueDetailScreen	Timeline, comments, upvotes, ratings
AnnouncementsScreen	Filtered announcement feed with priorities
BursaryStatusScreen	Bursary application status tracking
VoterRegistrationScreen	5-step voter registration with ID and selfie

---

5. Database Design

5.1 Database Strategy

The platform uses a **polyglot persistence** approach:

Database	Use Case
PostgreSQL	Structured data: admin users, bursary, areas, textbooks
MongoDB	Document data: constituents, issues, announcements
Redis	Session caching, rate limiting, performance optimization

5.2 PostgreSQL Schema (via Supabase)

Managed through migration scripts in `/migrations/`:

admin_users Table

```
CREATE TABLE admin_users (
    id           SERIAL PRIMARY KEY,
    username     VARCHAR(50) UNIQUE NOT NULL,
    password     VARCHAR(255) NOT NULL,  -- bcrypt hashed
    name         VARCHAR(100),
    phone        VARCHAR(20),
    role         VARCHAR(20) DEFAULT 'admin',  -- 'super_admin' or 'admin'
    created_at   TIMESTAMP DEFAULT NOW(),
    updated_at   TIMESTAMP DEFAULT NOW()
);
```

bursary_applications Table

```
CREATE TABLE bursary_applications (
    id              SERIAL PRIMARY KEY,
    ref_code        VARCHAR(20) UNIQUE NOT NULL,
    applicant_phone VARCHAR(20) NOT NULL,
    student_name    VARCHAR(100) NOT NULL,
    institution     VARCHAR(200),
    category        VARCHAR(50),
    amount          DECIMAL(10,2),
    status          VARCHAR(20) DEFAULT 'pending',
    admin_notes     TEXT,
    reviewed_by     VARCHAR(100),
    reviewed_at     TIMESTAMP,
    created_at      TIMESTAMP DEFAULT NOW(),
    updated_at      TIMESTAMP DEFAULT NOW()
);
```

areas Table

```
CREATE TABLE areas (
    id         SERIAL PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    type       VARCHAR(50),
    parent_id  INTEGER REFERENCES areas(id),
    created_at TIMESTAMP DEFAULT NOW()
);
```

textbook_requests / textbook_inventory / textbook_shortages Tables

```
-- Managed by migration: 05_textbook_system.sql
-- Tracks school textbook distribution and shortages
```

5.3 MongoDB Collections

constituents Collection

```
{
    "_id": "ObjectId",
    "phone_number": "+254 ... ",
    "full_name": "String",
    "national_id": "String",
    "name": "String (backward compat)",
    "location": "Kyamatu Ward",
    "area": "String",
    "registration_source": "USSD | Web | WhatsApp",
    "verification_status": "pending | verified | rejected",
    "created_at": "Date",
    "updated_at": "Date",
    "terms_accepted": true,
    "terms_accepted_at": "Date"
}
```

issues Collection

```
{
    "_id": "ObjectId",
    "ticket": "ISS-001",
    "category": "Roads & Infrastructure | Water & Sanitation | Security |  ... ",
    "message": "String (description)",
    "phone_number": "String",
    "location": "String",
    "status": "open | in_progress | resolved | closed",
    "source": "USSD | Dashboard | WhatsApp | Mobile",
    "photo_url": "String (optional)",
    "comments": ["Array of strings"],
    "upvotes": 0,
    "created_at": "Date",
    "updated_at": "Date"
}
```

announcements Collection

```
{
    "_id": "ObjectId",
    "title": "String",
    "body": "String",
    "category": "String",
    "priority": "normal | important | urgent",
    "views": 0,
    "created_at": "Date"
}
```

---

6. API Design

6.1 API Overview

All APIs follow RESTful conventions and return JSON responses (except WhatsApp webhook which returns TwiML XML).

6.2 Endpoint Summary

Health Endpoints

Method	Endpoint	Auth	Description
GET	`/health`	None	Basic health check
GET	`/health/detailed`	None	Memory, uptime, DB status

USSD Endpoint

Method	Endpoint	Auth	Description
POST	`/ussd`	None	Main USSD callback
POST	`/api/ussd`	None	Provider-agnostic USSD API

Admin Endpoints (via `/admin/api/`)

Method	Endpoint	Auth	Description
POST	`/admin/api/login`	None	Admin login (rate limited)
POST	`/admin/api/logout`	Token	Admin logout
GET	`/admin/api/announcements`	Token	List announcements
POST	`/admin/api/announcements`	Token	Create announcement
DELETE	`/admin/api/announcements/:id`	Token	Delete announcement
GET	`/admin/api/issues`	Token	List issues
PUT	`/admin/api/issues/:id/status`	Token	Update issue status
GET	`/admin/api/users`	Token	List admin users
POST	`/admin/api/users`	MCA	Create admin user
DELETE	`/admin/api/users/:id`	MCA	Delete admin user
GET	`/admin/api/constituents`	Token	List constituents
PUT	`/admin/api/constituents/:id/verify`	MCA	Verify constituent
GET	`/admin/api/bursaries`	Token	List bursary applications
PUT	`/admin/api/bursaries/:id/status`	Token	Update bursary status
GET	`/admin/api/export/constituents`	Token	Export constituents CSV
GET	`/admin/api/export/issues`	Token	Export issues CSV
GET	`/admin/api/export/bursaries`	Token	Export bursary CSV

Citizen Portal Endpoints

Method	Endpoint	Auth	Description
POST	`/api/citizen/request-otp`	None	Request login OTP
POST	`/api/citizen/verify-otp`	None	Verify OTP, get token
GET	`/api/citizen/issues`	Bearer	Get citizen's issues
POST	`/api/citizen/issues`	Bearer	Submit new issue
GET	`/api/citizen/bursaries`	Bearer	Get bursary applications

WhatsApp Endpoint

Method	Endpoint	Auth	Description
POST	`/api/whatsapp/webhook`	Twilio	Receive WhatsApp messages

6.3 Request/Response Examples

Login Request:

```
POST /admin/api/login
{
    "username": "Martin",
    "password": "Martin@21"
}
```

Login Response:

```
{
    "success": true,
    "token": "a1b2c3d4e5f6 ... ",
    "user": {
        "id": 1,
        "name": "Martin",
        "role": "super_admin"
    }
}
```

USSD Callback Request:

```
POST /ussd
{
    "sessionId": "ATSessionId123",
    "phoneNumber": "+254712345678",
    "text": "1*John Doe*12345678",
    "serviceCode": "*384#"
}
```

---

7. Security Design

7.1 Authentication Mechanisms

Channel	Method
Admin Dashboard	Username + 6-digit PIN → bcrypt verification → Session token
Citizen Portal	Phone number → OTP → Bearer token (24h expiry)
USSD	Phone number (MSISDN) from telecom gateway
WhatsApp	Phone number from Twilio webhook

7.2 Security Middleware Stack

The `SecurityMiddleware` class (`middleware/security.js`) provides:

1. **Global Rate Limiter** — 100 requests/second
2. **USSD Rate Limiter** — Telecom-specific limits
3. **Admin Rate Limiter** — Admin endpoint protection
4. **Auth Rate Limiter** — Login brute-force prevention
5. **Input Validation** — Sanitize all request bodies
6. **Suspicious Activity Detection** — Pattern-based threat detection
7. **Progressive Blocking** — Escalating penalties for repeated violations

7.3 Security Headers

```
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
```

7.4 Data Protection

- Passwords hashed with bcrypt (10 rounds)
- Session tokens generated using `crypto.randomBytes(32)`
- OTPs expire after 5 minutes
- Admin sessions expire after 30 minutes of inactivity
- Environment variables for all secrets (API keys, DB credentials)
- Phone number anonymization for logs (`privacy.js`)
- CORS origin validation

7.5 Access-Gated Rollout

The USSD module supports an allowlisted MSISDN set for controlled rollout:

```
const ALLOWED_MSISDNS = new Set(["+254114945842"]);
```

---

8. User Interface Design

8.1 USSD Interface

USSD menus use numbered prompts (max 160 characters per screen):

```
KYAMATU WARD – FREE SERVICE

Select Language:
1. English
2. Swahili
3. Kamba
```

8.2 Admin Dashboard

The admin dashboard is a server-rendered web application served from `/admin`:

- Responsive design for desktop and tablet
- Session-based authentication
- Real-time data display with pagination
- CSV export buttons for reporting

8.3 Mobile App

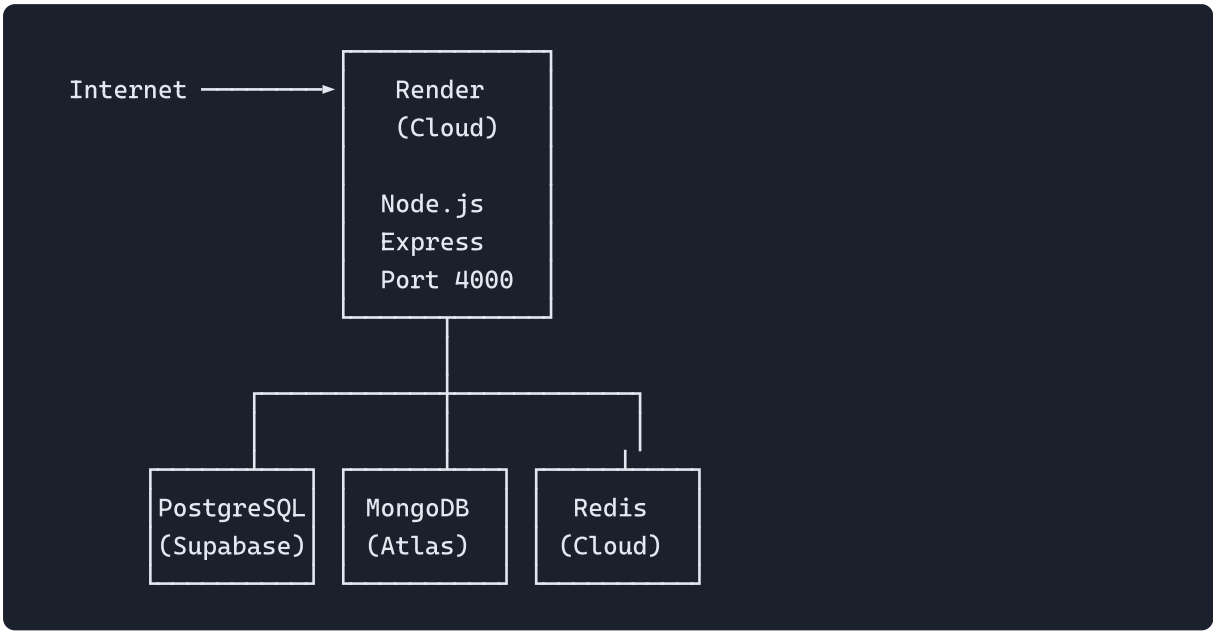
The React Native app features:

- Material Design-inspired UI
- 5 feature cards on the dashboard
- Camera integration for issue photos and voter ID verification
- GPS location tagging
- Push notification support (planned)

---

9. Deployment Architecture

9.1 Production Deployment



9.2 Configuration

All configuration is managed via environment variables:

Variable	Purpose
`PORT`	Server port (default: 4000)
`DATABASE_URL`	PostgreSQL connection string
`REDIS_URL`	Redis connection string
`JWT_SECRET`	JWT signing secret
`OPENAI_API_KEY`	OpenAI API key for AI features
`TWILIO_ACCOUNT_SID`	Twilio WhatsApp integration
`TWILIO_AUTH_TOKEN`	Twilio authentication
`SENTRY_DSN`	Error monitoring (Sentry)
`FRONTEND_URL`	CORS allowed origin

9.3 Docker Support

```
# Dockerfile included for containerized deployment
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 4000
CMD ["node", "src/index.js"]
```

9.4 Health Monitoring

- `GET /health` — Fast liveness probe (no DB dependency)
- `GET /health/detailed` — Deep health check with memory, uptime, and DB status
- Sentry integration for production error tracking

---

10. Technology Stack

10.1 Backend Technologies

Technology	Version	Purpose
Node.js	18+	Server runtime
Express.js	5.1.0	Web framework
PostgreSQL	15+	Relational database (via Supabase)
MongoDB	6.x	Document database
Redis	4.x	Caching and session store
bcryptjs	2.4.3	Password hashing
twilio	4.20.0	WhatsApp integration
openai	4.24.0	AI assistant integration
sharp	0.32.5	Image processing
@sentry/node	7.91.0	Error monitoring
morgan	1.10.0	HTTP request logging
compression	1.7.4	Response compression

10.2 Mobile App Technologies

Technology	Purpose
React Native	Cross-platform mobile framework
Expo	Development toolchain
React Navigation	Screen navigation

10.3 DevOps & Infrastructure

Tool	Purpose
Docker	Containerization
Render	Cloud hosting platform
Supabase	Managed PostgreSQL
GitHub	Version control
Sentry	Error monitoring & alerting

End of Design Specifications Document

VOO Ward Citizen Engagement Platform — Confidential Document